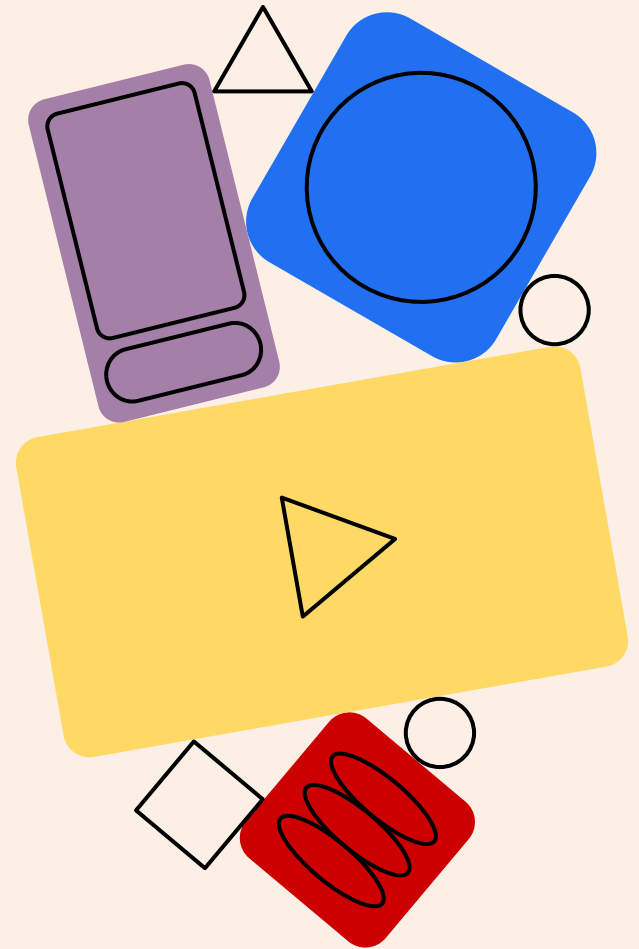


# -Hashing -HashTable



# What is Hashing

- Hashing is a technique used for storing and retrieving information as quickly as possible.
- It is used to perform optimal searches and is useful in implementing symbol tables.

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|   | D |   | Z |   |   | C | Q |   |   |    |

- In the Trees chapter we saw that balanced binary search trees support operations such as insert, delete and search in  $O(\log n)$  time.
- In applications, if we need these operations in  $O(1)$ , then hashing provides a way.
- Remember that worst case complexity of hashing is still  $O(n)$ , but it gives  $O(1)$  on the average.

# ADT

- The common operations for hash table are:
  - **CreatHashTable:** Creates a new hash table
  - **HashSearch:** Searches the key in hash table
  - **HashInsert:** Inserts a new key into hash table
  - **HashDelete:** Deletes a key from hash table
  - **DeleteHashTable:** Deletes the hash table

# Introduction

- In simple terms we can treat array as a hash table. For understanding the use of hash tables, let us consider the following example: Give an algorithm for printing the first repeated character if there are duplicated elements in it.
- The simple and brute force way of solving is: given a string, for each character check whether that character is repeated or not.
- The time complexity of this approach is  $O(n^2)$  with  $O(1)$  space complexity.
- Now, let us find a better solution for this problem. Since our objective is to find the first repeated character, what if we remember the previous characters in some array?

# Introduction

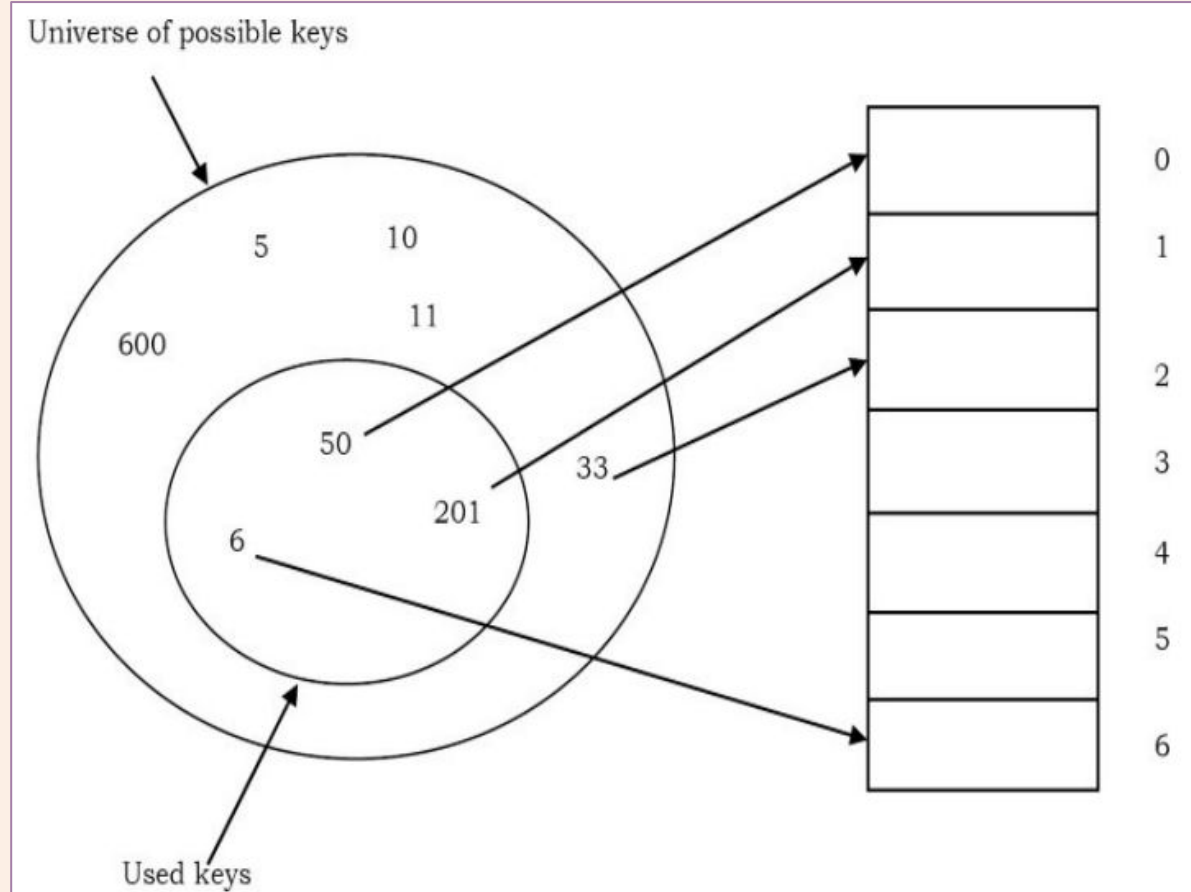
- We know that the number of possible characters is 256 (for simplicity assume ASCII characters only).
- Create an array of size 256 and initialize it with all zeros. For each of the input characters go to the corresponding position and increment its count. Since we are using arrays, it takes constant time for reaching any location.
- While scanning the input, if we get a character whose counter is already 1 then we can say that the character is the one which is repeating for the first time.
- **Now, let us consider a slight variant of the same problem. Suppose the given array has numbers instead of characters, then how do we solve the problem?**

# Introduction

- In this case the set of possible values is infinity (or at least very big).
- Creating a huge array and storing the counters is not possible.
- That means there are a set of universal keys and limited locations in the memory.
- If we want to solve this problem we need to somehow map all these possible keys to the possible memory locations.
- We need a mapping of possible keys to one of the available locations.
- The process of mapping the keys to locations is called **hashing**.
- **Note:** *Keys are mapped to locations. That depends on the function used for conversions. One such simple function is  $\text{key \% table size}$ .*

# Introduction

- Illustration – Hashing



# Components of Hashing

- Hashing has four key components:
  - 1) Hash Table
  - 2) Hash Functions
  - 3) Collisions
  - 4) Collision Resolution Techniques



# Hash Table

- Hash table is a generalization of array. With an array, we store the element whose key is  $k$  at a position  $k$  of the array. That means, given a key  $k$ , we find the element whose key is  $k$  by just looking in the  $k$ th position of the array. This is called direct addressing.
- Direct addressing is applicable when we can afford to allocate an array with one position for every possible key. But if we do not have enough space to allocate a location for each possible key, then we need a mechanism to handle this case.
- Another way of defining the scenario is: if we have less locations and more possible keys, then simple array implementation is not enough.

# Hash Table

- In these cases one option is to use hash tables.
- Hash table or hash map is a data structure that stores the keys and their associated values, and hash table uses a hash function to map keys to their associated values.
- The general convention is that we use a hash table when the number of keys actually stored is small relative to the number of possible keys.

# Hash Function

- The hash function is used to transform the key into the index.
- Ideally, the hash function should map each possible key to a unique slot index, but it is difficult to achieve in practice.
- Given a collection of elements, a hash function that maps each item into a unique slot is referred to as a perfect hash function.
- If we know the elements and the collection will never change, then it is possible to construct a perfect hash function.
- Unfortunately, given an arbitrary collection of elements, there is no systematic way to construct a perfect hash function.
- Luckily, we do not need the hash function to be perfect to still gain performance efficiency.

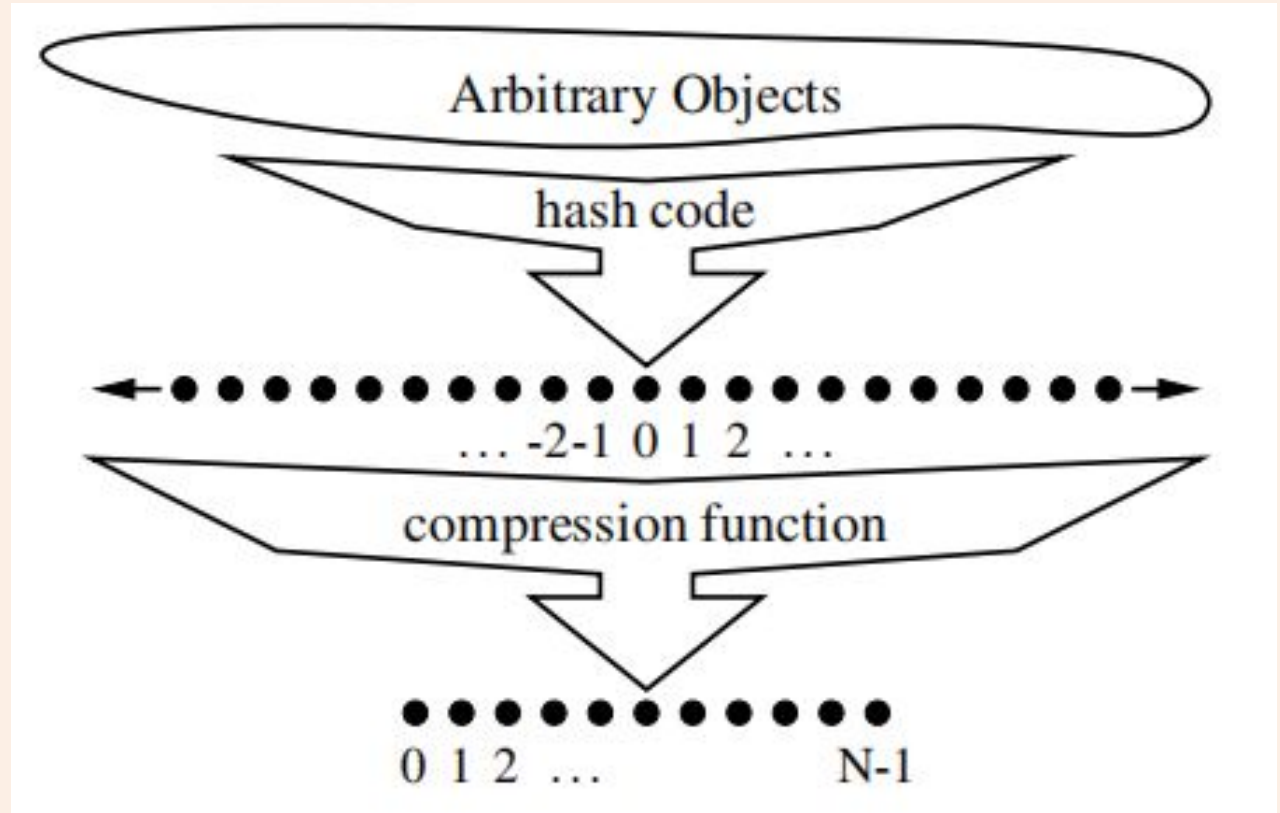
# Hash Function

- One way to always have a perfect hash function is to increase the size of the hash table so that each possible value in the element range can be accommodated.
- This guarantees that each element will have a unique slot.
- Although this is practical for small numbers of elements, it is not feasible when the number of possible elements is large. For example, if the elements were nine-digit/ twelve-digit
- Social Security/ AADHAR numbers, this method would require more than billions slots.
- If we only want to store data for a class of 25 students, we will be wasting an enormous amount of memory.
- Our goal is to create a hash function that minimizes the number of collisions, is easy to compute, and evenly distributes the elements in the hash table. There are a number of common ways to extend the simple remainder method. We will consider a few of them here.

# Hash Function

- The folding method for constructing hash functions begins by dividing the elements into equal size pieces (the last piece may not be of equal size).
- These pieces are then added together to give the resulting hash value. For example, if our element was the phone number 436-555-4601, we would take the digits and divide them into groups of 2 (43,65,55,46,01).
- After the addition,  $43+65+55+46+01$ , we get 210. If we assume our hash table has 11 slots, then we need to perform the extra step of dividing by 11 and keeping the remainder.
- In this case  $210 \% 11$  is 1, so the number 436-555-4601 hashes to slot 1.
- Some folding methods go one step further and reverse every other piece before the addition. For the above example, we get  $43+56+55+64+01=219$  which gives  $219 \% 11 = 10$ .

# Hash Function



Compression:  
Division  
Mul-Add-Divide(MAD)

# How to Choose Hash Function?

- The basic problems associated with the creation of hash tables are:
  - An efficient hash function should be designed so that it distributes the index values of inserted objects uniformly across the table.
  - An efficient collision resolution algorithm should be designed so that it computes an alternative index for a key whose hash index corresponds to a location previously inserted in the hash table.
  - We must choose a hash function which can be calculated quickly, returns values within the range of locations in our table, and minimizes collisions.

# Good Hash Functions

- Characteristics of Good Hash Functions
  - Minimize collision
  - Be easy and quick to compute
  - Distribute key values evenly in the hash table
  - Use all the information provided in the key
  - Have a high load factor for a given set of keys



# Load Factor

- The load factor of a non-empty hash table is the number of items stored in the table divided by the size of the table.
- This is the decision parameter used when we want to rehash or expand the existing hash table entries.
- This also helps us in determining the efficiency of the hashing function.
- That means, it tells whether the hash function is distributing the keys uniformly or not

$$\text{Load factor} = \frac{\text{Number of elements in hash table}}{\text{Hash Table size}}$$

# Collisions

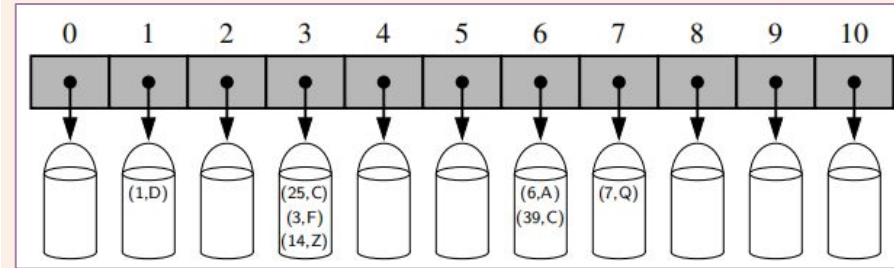
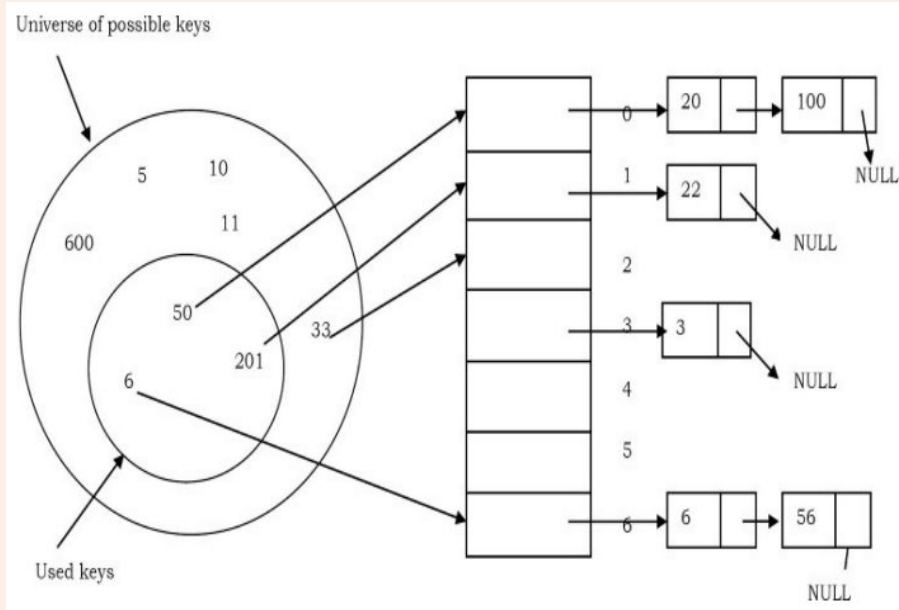
- Hash functions are used to map each key to a different address space, but practically it is not possible to create such a hash function and the problem is called collision.
- Collision is the condition where two records are stored in the same location

# Collision Resolution Techniques

- The process of finding an alternate location is called collision resolution.
- Even though hash tables have collision problems, they are more efficient in many cases compared to all other data structures, like search trees.
- There are a number of collision resolution techniques, and the most popular are direct chaining and open addressing.
- **Direct Chaining:** An array of linked list application
  - Separate chaining
- **Open Addressing:** Array-based implementation
  - Linear probing (linear search)
  - Quadratic probing (nonlinear search)
  - Double hashing (use two hash functions)

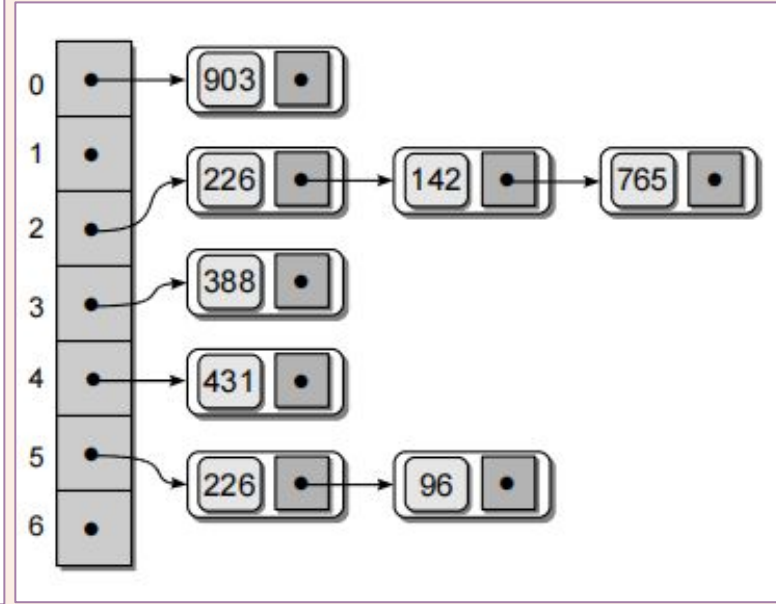
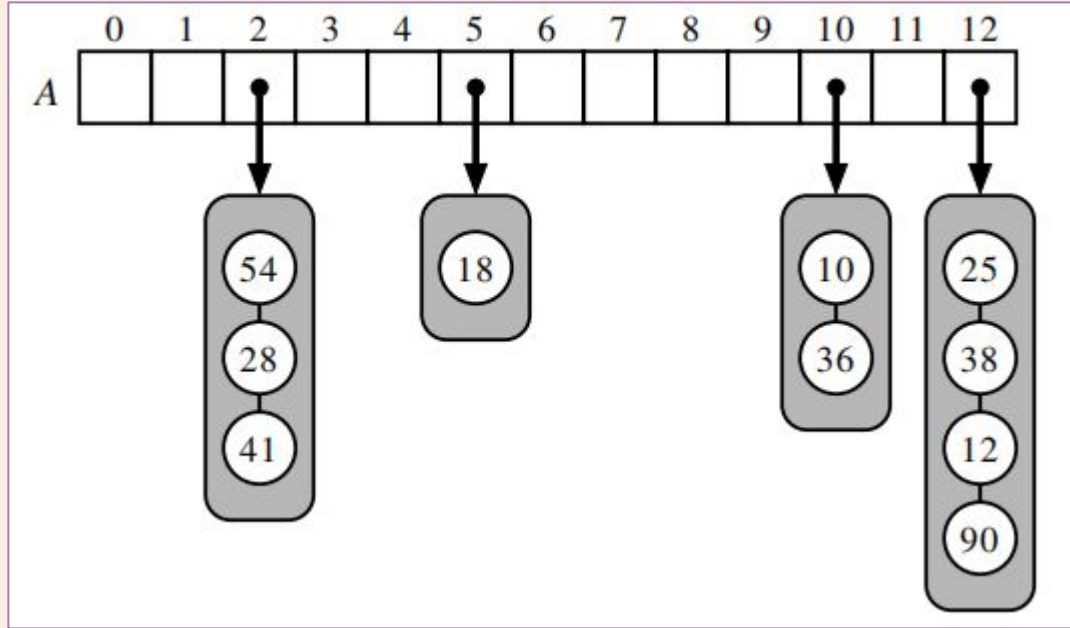
# Separate Chaining

- Collision resolution by chaining combines linked representation with hash table. When two or more records hash to the same location, these records are constituted into a singly-linked list called a chain.



# Separate Chaining

- Collision resolution by chaining combines linked representation with hash table. When two or more records hash to the same location, these records are constituted into a singly-linked list called a chain.



# Open Addressing

- In open addressing all keys are stored in the hash table itself. This approach is also known as closed hashing. This procedure is based on probing. A collision is resolved by probing.
- Open addressing requires that the load factor is always at most 1 and that items are stored directly in the cells of the bucket array itself.

# Linear Probing

- The interval between probes is fixed at 1. In linear probing, we search the hash table sequentially, starting from the original hash location.
- If a location is occupied, we check the next location.
- We wrap around from the last table location to the first table location if necessary. The function for rehashing is the following:  
**`rehash(key) = (n + 1) % tablesize`**
- One of the problems with linear probing is that table items tend to cluster together in the hash table. This means that the table contains groups of consecutively occupied locations that are called clustering.
- Clusters can get close to one another, and merge into a larger cluster. Thus, the one part of the table might be quite dense, even though another part has relatively few items.
- Clustering causes long probe searches and therefore decreases the overall efficiency.

# Linear Probing

- The next location to be probed is determined by the step-size, where other step-sizes (more than one) are possible.
- The step-size should be relatively prime to the table size, i.e. their greatest common divisor should be equal to 1.
- If we choose the table size to be a prime number, then any step-size is relatively prime to the table size.
- Clustering cannot be avoided by larger step-sizes.



# Quadratic Probing

- The interval between probes increases proportionally to the hash value (the interval thus increasing linearly, and the indices are described by a quadratic function).
- The problem of Clustering can be eliminated if we use the quadratic probing method.
- In quadratic probing, we start from the original hash location  $i$ . If a location is occupied, we check the locations  $i + 1^2, i + 2^2, i + 3^2, i + 4^2$
- ... We wrap around from the last table location to the first table location if necessary.
- The function for rehashing is the following:  
**`rehash(key) = (n + k2) % tablesize`**

# Quadratic Probing

- Example: Let us assume that the table size is 11 (0..10)
- Hash Function:  $h(\text{key}) = \text{key} \bmod 11$

Insert keys

- $31 \bmod 11 = 9$
- $19 \bmod 11 = 8$
- $2 \bmod 11 = 2$
- $13 \bmod 11 = 2 \rightarrow 2 + 1^2 = 3$
- $25 \bmod 11 = 3 \rightarrow 3 + 1^2 = 4$
- $24 \bmod 11 = 2 \rightarrow 2 + 1^2, 2 + 2^2 = 6$
- $21 \bmod 11 = 10$
- $9 \bmod 11 = 9 \rightarrow 9 + 1^2, 9 + 2^2 \bmod 11, 9 + 3^2 \bmod 11 = 7$

|    |    |
|----|----|
| 0  |    |
| 1  |    |
| 2  | 2  |
| 3  | 13 |
| 4  | 25 |
| 5  | 5  |
| 6  | 24 |
| 7  | 9  |
| 8  | 19 |
| 9  | 31 |
| 10 | 21 |

# Quadratic Probing

- Even though clustering is avoided by quadratic probing, still there are chances of clustering.
- Clustering is caused by multiple search keys mapped to the same hash key.
- Thus, the probing sequence for such search keys is prolonged by repeated conflicts along the probing sequence.
- Both linear and quadratic probing use a probing sequence that is independent of the search key.

# Double Hashing

- The interval between probes is computed by another hash function.
- Double hashing reduces clustering in a better way.
- The increments for the probing sequence are computed by using a second hash function. The second hash function  $h_2$  should be:  
 **$h_2(\text{key}) \neq 0$  and  $h_2 \neq h_1$**
- We first probe the location  $h_1(\text{key})$ . If the location is occupied, we probe the location  $h_1(\text{key}) + h_2(\text{key})$ ,  $h_1(\text{key}) + 2 * h_2(\text{key})$ , ...

# Double Hashing

- Table size is 11 (0..10)
- **Hash Function:** assume  
 $h1(key) = key \bmod 11$  and  $h2(key) = 7 - (key \bmod 7)$
- Insert keys:

$$58 \bmod 11 = 3$$

$$14 \bmod 11 = 3 \rightarrow 3 + 7 = 10$$

$$91 \bmod 11 = 3 \rightarrow 3 + 7, 3 + 2 * 7 \bmod 11 = 6$$

$$25 \bmod 11 = 3 \rightarrow 3 + 3, 3 + 2 * 3 = 9$$

|    |    |
|----|----|
| 0  |    |
| 1  |    |
| 2  |    |
| 3  | 58 |
| 4  | 25 |
| 5  |    |
| 6  | 91 |
| 7  |    |
| 8  |    |
| 9  | 25 |
| 10 | 14 |

# Open Addressing vs. Separate Chaining

- It is somewhat complicated because we have to account for the memory usage.
- Separate chaining uses extra memory for links.
- Open addressing needs extra memory implicitly within the table to terminate the probe sequence.
- Open-addressed hash tables cannot be used if the data does not have unique keys.
- An alternative is to use separate chained hash tables.

# Linear Probing vs. Double Hashing

- The choice between linear probing and double hashing depends on the cost of computing the hash function and on the load factor [number of elements per slot] of the table.
- Both use few probes but double hashing take more time because it hashes to compare two hash functions for long keys.

# Comparisons - Open Addressing

| Linear Probing                                 | Quadratic Probing                                                            | Double hashing                                               |
|------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| Fastest among three                            | Easiest to implement and deploy                                              | Makes more efficient use of memory                           |
| Uses few probes                                | Uses extra memory for links and it does not probe all locations in the table | Uses few probes but takes more time                          |
| A problem occurs known as primary clustering   | A problem occurs known as secondary clustering                               | More complicated to implement                                |
| Interval between probes is fixed - often at 1. | Interval between probes increases proportional to the hash value             | Interval between probes is computed by another hash function |





**Thank You**